

MÓDULO DE FORMACIÓN

GitHub Copilot

Características avanzadas

Texto fantasma, chat, comandos de barra diagonal y agentes aplicados a un proyecto real en GitHub Codespaces

Guía práctica · Python · Visual Studio Code

Un asistente de codificación con IA

GitHub Copilot proporciona sugerencias de autocompletado mientras se programa, a partir del código ya escrito o mediante lenguaje natural.

Analiza el archivo actual y los archivos relacionados, y utiliza el contexto del código y de los comentarios para sugerir nuevas líneas o funciones completas.

GitHub Codespaces

Entorno de desarrollo hospedado en la nube que puede ejecutarse con Visual Studio Code. Permite personalizar dependencias, bibliotecas, extensiones y configuración del editor para cada proyecto de GitHub.



Trabajar con un proyecto existente

Copilot aumenta la productividad tanto en proyectos nuevos como existentes, aplicando funciones avanzadas de IA a tareas cotidianas.

Escritura de código

Generación de líneas y funciones completas.

Documentación

Redacción y actualización de documentos del proyecto.

Pruebas

Creación de pruebas unitarias sobre el código existente.

Puntos de conexión de API

Incorporación de nuevas rutas HTTP.

Comprensión del código

Explicación de un código base ya existente.

El módulo utiliza ejemplos prácticos para modificar un repositorio real mediante distintas técnicas de Copilot.

Al finalizar el módulo se podrá

01

Trabajar con un repositorio de GitHub preconfigurado en Codespaces con la extensión GitHub Copilot.

02

Utilizar características interactivas de Copilot para generar sugerencias útiles en un proyecto existente.

03

Aplicar características avanzadas para comprender un proyecto, escribir documentación y crear pruebas unitarias.

04

Utilizar avisos interactivos y otras funciones avanzadas para mejorar un proyecto de software.

Qué se necesita antes de empezar

Conocimientos técnicos

Python y editores de texto a nivel básico. Comprensión de Git y GitHub Fundamentals, incluidos comandos como `git add` y `git push`.

Cuenta y suscripción

Cuenta de GitHub con una suscripción activa a GitHub Copilot: personal o administrada por una organización o empresa.

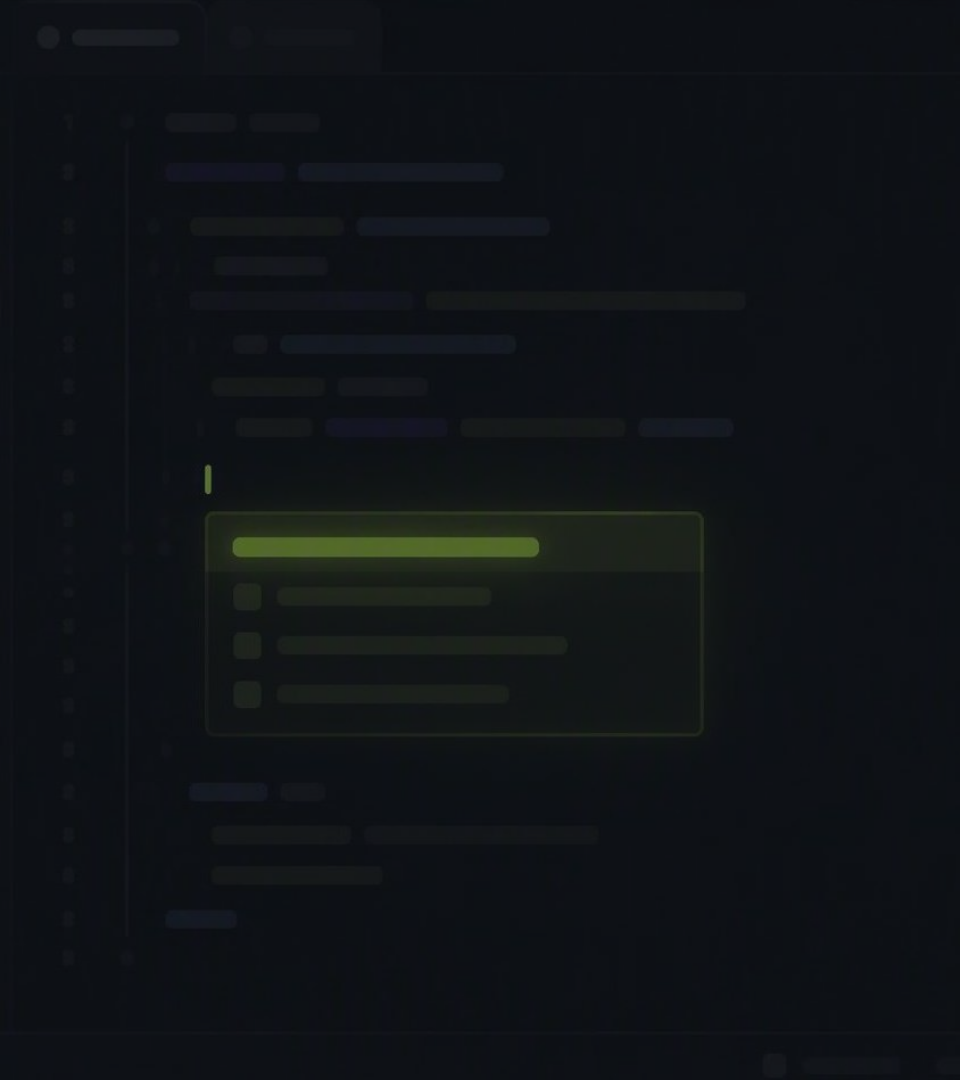
Para aprendizaje, Copilot Free es suficiente

El nivel gratuito incluye límites de uso mensuales y permite completar los ejercicios del módulo sin contratar una suscripción de pago.

SECCIÓN 01

Características avanzadas de GitHub Copilot

Consultar documentación, comprender el código base, corregir errores y escribir pruebas consume mucho tiempo. Copilot ofrece funciones para facilitar estas tareas.

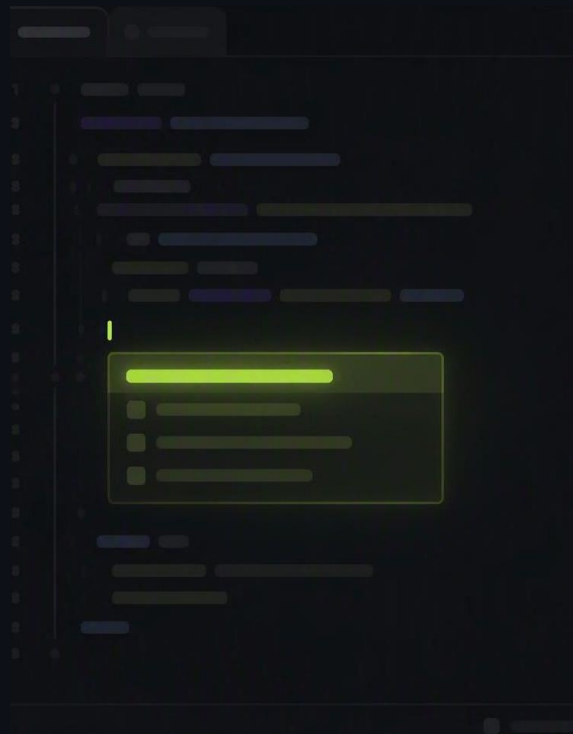


Texto fantasma

Cuando Copilot está habilitado, muestra sugerencias en línea denominadas texto fantasma.

- La sugerencia puede ignorarse simplemente al seguir escribiendo.
- Se acepta presionando la tecla Tabulador.
- No hace falta escribir una indicación: Copilot usa de forma predeterminada los archivos abiertos como contexto.
- También se puede solicitar mediante comentarios, la ventana de chat o el chat en línea.

```
def calcular_total(items):  
    # Copilot sugiere la implementación → Tab
```



Chat y chat en línea

Chat con GitHub Copilot

Conversación interactiva desde una interfaz dedicada. En Visual Studio Code se abre con el icono de chat de la barra lateral izquierda para preguntar sobre el código en el que se trabaja u otras cuestiones de software.

Chat en línea

Permite interactuar con Copilot directamente desde el código, sin cambiar de contexto. Las sugerencias aparecen junto al código, evitando saltar a otro panel.

Accesos directos del chat en línea

Windows

Ctrl + i

Mac

Command + i

Comandos de barra diagonal y agentes

Al escribir / en el panel de chat o en el chat en línea aparece un menú con los comandos disponibles. Indican una intención específica y resuelven tareas comunes sin escribir mensajes extensos.

/fix

Corrige problemas en el código

/doc

Agrega comentarios al código seleccionado

/explain

Explica el código

/generate

Genera código para una solicitud

/help

Ayuda sobre el uso del chat

/optimize

Analiza y mejora el tiempo de ejecución

/tests

Crea pruebas unitarias

@terminal

Agente: contexto del terminal

Los agentes de Visual Studio Code permiten interactuar con Copilot usando un contexto específico, como la salida del terminal con `@terminal`.

Configuración de Copilot en Visual Studio Code

Se crea un repositorio a partir de una plantilla de GitHub para una API web desarrollada con Python, usando un Codespace preconfigurado con la extensión de Copilot.

- 1 Abrir el Codespace con el entorno preconfigurado.
- 2 En Crear codespace, revisar las opciones y seleccionar Crear nuevo codespace.
- 3 Esperar a que se inicie; el proceso puede tardar unos minutos.
- 4 Realizar los demás ejercicios dentro de este contenedor de desarrollo.

Codespaces gratuito

Todas las cuentas de GitHub disponen de hasta 60 horas gratuitas al mes con dos instancias principales.

Registro en Copilot

Si aún no se ha hecho, hay que registrarse con una prueba gratuita o una suscripción. Educadores, estudiantes y mantenedores de código abierto pueden hacerlo sin coste.

Copilot Free y la API web de Python

2000

autocompletados de código al mes

50

mensajes de chat al mes

8000

puerto del servidor en Codespace

Puesta en marcha

- Codespaces carga una sección de terminal en la parte inferior.
- Instala en el contenedor las extensiones necesarias.
- Tras instalar los paquetes, ejecuta `uvicorn` para iniciar la aplicación.
- La pestaña Puertos muestra el servidor en ejecución.

Cómo activar Copilot Free

Abrir Visual Studio Code, seleccionar el icono de GitHub Copilot y elegir «Iniciar sesión para usar GitHub Copilot de forma gratuita». Educadores, estudiantes y algunos mantenedores de código abierto pueden recibir Copilot Pro sin coste.

Mensajes implícitos

Además de escribir mensajes específicos, se pueden usar características que aprovechan implícitamente el contexto disponible para obtener respuestas.

Ejemplo: corregir código con error

- Seleccionar el código Python con el error.
- Abrir el chat en línea con Ctrl+i o Cmd+i.
- Escribir /fix para solicitar la corrección.

```
/fix
```

```
> Copilot propone la corrección  
> sobre la selección actual
```

Dónde se usan

Los comandos de barra diagonal funcionan tanto en el chat en línea como en la interfaz de chat, así que se elige la forma de interacción más adecuada según el código y la tarea.

Combinar es la clave

Un comando específico junto con el contexto adecuado ofrece mejores respuestas que un mensaje largo y genérico.

Contexto selectivo

Copilot puede ofrecer sugerencias según el contexto en el que se trabaja, sin necesidad de abrir numerosos archivos.

Todo el área de trabajo

La salida del terminal

Los archivos abiertos en el editor

Ejemplos de solicitud

```
I need to create a Dockerfile for this project, can you generate one that will help me package it?
```

```
Help me create a Dockerfile to package this project but make sure you are using a Virtual Environment for Python.
```

```
@terminal ¿Cómo se corrige el mensaje de error que veo?
```

Actualizar una API web con Copilot

El repositorio contiene una aplicación web de Python que hospeda una API de Tiempo de Viajes. La API no expone el país o región necesario para enumerar ciudades: hay que implementar una ruta GET que devuelva JSON con niveles históricos altos y bajos para un país, ciudad y mes determinados.

Paso 1 · Agregar una nueva ruta

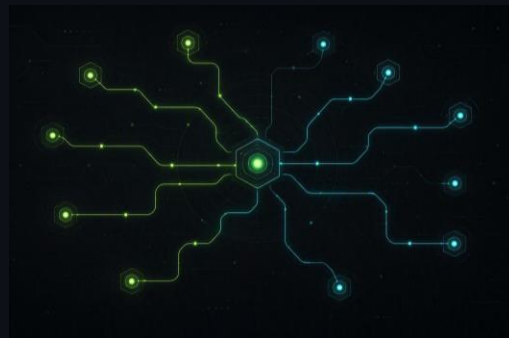
Abrir `main.py` y usar el chat en línea: «Create a new route that exposes the cities of a country/region». Probar la ruta y perfeccionar el mensaje.

Paso 2 · Crear una prueba

Usar `/tests` para generar una prueba que utilice España como país o región, y aportar detalles adicionales si el resultado no es correcto.

Paso 3 · Documentar con un agente

Abrir `README.md` y pedir al modo agente que documente cómo ejecutar el proyecto para otros desarrolladores.



```
@app.get(
    '/countries/{country}')
def cities(country: str):
    return list(
        data[country].keys())
```

RESULTADO DEL EJERCICIO

Qué se logró con GitHub Copilot

01

Generar una nueva ruta de API

02

Crear una prueba para comprobar su exactitud

03

Agregar documentación mediante un agente

A continuación: una prueba breve de conocimientos, un resumen de lo aprendido y el distintivo por completar el módulo.